

onebladeTNflex

May 14, 2026

```
[1]: import numpy as np                                # numerical arrays and
      ↪ operations
import sympy as sp                                    # symbolic math engine
from sympy.physics.mechanics import * # ReferenceFrame, Point, Lagrangian, etc.
import scipy.linalg as la                            # eigenvalues, linear solvers
from scipy.integrate import solve_ivp                 # Runge-Kutta ODE integrator
import matplotlib.pyplot as plt                       # 2D plotting
init_vprinting()                                     # render SymPy output as
      ↪ LaTeX in notebook
```

1 3 DOF

1.1 Tilting Nacelle, Single Flexible Blade

```
[2]: # Symbolic variables
theta, psi, q    = dynamicsymbols('theta psi q')      # DOFs: nacelle tilt, rotor
      ↪ azimuth, blade flap
t                = sp.symbols('t')                    # independent time variable
Omega            = sp.Symbol('Omega', real=True)       # symbolic steady-state
      ↪ rotor speed

m_n, m_b         = sp.symbols('m_n m_b')              # nacelle and blade masses
k_t, c_t, g      = sp.symbols('k_t c_t g')            # tower stiffness, damping,
      ↪ gravity
X_n, Z_n         = sp.symbols('X_n Z_n')              # nacelle COM position in
      ↪ tower frame
X_h, Z_h         = sp.symbols('X_h Z_h')              # hub location in tower
      ↪ frame
Z_b              = sp.symbols('Z_b')                  # blade COM radial offset
      ↪ in rotor frame
J_xx_n, J_yy_n, J_zz_n = sp.symbols('J_xx_n J_yy_n J_zz_n') # nacelle inertia
      ↪ components
J_xx_b, J_yy_b, J_zz_b = sp.symbols('J_xx_b J_yy_b J_zz_b') # blade inertia
      ↪ components

GM_b, K_b        = sp.symbols('GM_b K_b')             # blade modal mass and
      ↪ modal stiffness
```

```
M_phi, M_rphi = sp.symbols('M_phi M_rphi') # coupling integrals:
↳  $\varphi(r)dm$  and  $r\varphi(r)dm$ 
```

```
[3]: # Reference frames and kinematics
N = ReferenceFrame('N') # inertial ground frame
T = N.orientnew('T', 'Axis', [theta, N.y]) # tower/nacelle frame:
↳ tilts about N.y
R = T.orientnew('R', 'Axis', [psi, T.x]) # rotor frame: spins about
↳ T.x

O = Point('O') # fixed reference point at
↳ tower top
NCOM = O.locatenew('NCOM', X_n*T.x + Z_n*T.z) # nacelle COM position
Hub = O.locatenew('Hub', X_h*T.x + Z_h*T.z) # hub center position
BCOM_rigid = Hub.locatenew('BCOM_rigid', Z_b*R.z) # undeformed blade COM
↳ position

O.set_vel(N, 0) # tower top is fixed in
↳ inertial frame
NCOM.v2pt_theory(O, N, T) # nacelle COM velocity via
↳ two-point theorem
Hub.v2pt_theory(O, N, T) # hub velocity via
↳ two-point theorem
BCOM_rigid.v2pt_theory(Hub, N, R) # undeformed blade COM
↳ velocity
```

[3]: $Z_h\dot{\theta}\hat{\mathbf{t}}_x - X_h\dot{\theta}\hat{\mathbf{t}}_z + Z_b\cos(\psi)\dot{\theta}\hat{\mathbf{r}}_x - Z_b\dot{\psi}\hat{\mathbf{r}}_y$

```
[4]: # Inertias and Rigid Bodies
J_nac = inertia(T, J_xx_n, J_yy_n, J_zz_n) # nacelle inertia dyadic in
↳ tower frame
J_blade = inertia(R, J_xx_b, J_yy_b, J_zz_b) # blade inertia dyadic in
↳ rotor frame

Nacelle = RigidBody('Nacelle', NCOM, T, m_n, (J_nac, NCOM)) #
↳ nacelle rigid body
Blade = RigidBody('Blade', BCOM_rigid, R, m_b, (J_blade, BCOM_rigid)) #
↳ blade rigid body
```

```
[5]: # Energies, Forces
# Potential Energies (Gravity + Strain)
Nacelle.potential_energy = (m_n*g*NCOM.pos_from(O).dot(N.z) # nacelle
↳ gravitational PE
+ sp.Rational(1,2)*k_t*theta**2) # tower torsional
↳ spring PE
Blade.potential_energy = m_b*g*BCOM_rigid.pos_from(O).dot(N.z) # blade
↳ gravitational PE
```

```

V_flex = sp.Rational(1, 2) * K_b * q**2 # blade flapwise
↳ strain PE

# Flexible Kinetic Energy and FFRF Coupling
T_flex = sp.Rational(1, 2) * GM_b * q.diff(t)**2 # blade flapwise
↳ kinetic energy

v_hub = Hub.vel(N)
omega_b = R.ang_vel_in(N)
T_coupling = (q.diff(t) * M_phi * v_hub.dot(R.x) # flex
↳ translation coupling
               + q.diff(t) * M_rphi * omega_b.dot(R.y)) # flex rotation
↳ coupling

damping = [(T, -c_t*theta.diff(t)*N.y)] # viscous tower
↳ damping torque

# display(Lagrangian(N, Nacelle))
# display(Lagrangian(N, Blade))

```

```

[6]: # Equations of Motion via Lagrange's Method
L_rigid = Lagrangian(N, Nacelle, Blade) # rigid body
↳ lagrangian

L = L_rigid + T_flex + T_coupling - V_flex # total system
↳ lagrangian

print("Lagrangian:")
display(L) # Display the total Lagrangian
LM = LagrangesMethod(L, [theta, psi, q], forcelist=damping, frame=N)
EOM = LM.form_lagranges_equations() # generate the symbolic EOMs
print("Equations of Motion:")
display(EOM)

```

Lagrangian:

$$\begin{aligned}
& \frac{GM_b \dot{q}^2}{2} + \frac{J_{xxb} \dot{\psi}^2}{2} + \frac{J_{yyb} \dot{\theta}^2}{2} - \frac{K_b q^2}{2} + M_\phi Z_h \dot{q} \dot{\theta} + M_{rphi} \cos(\psi) \dot{q} \dot{\theta} - \\
& gm_b (-X_h \sin(\theta) + Z_b \cos(\psi) \cos(\theta) + Z_h \cos(\theta)) - gm_n (-X_n \sin(\theta) + Z_n \cos(\theta)) - \\
& \frac{k_t \theta^2}{2} + \frac{m_b (X_h^2 \dot{\theta}^2 + 2X_h Z_b \sin(\psi) \dot{\psi} \dot{\theta} + Z_b^2 \cos^2(\psi) \dot{\theta}^2 + Z_b^2 \dot{\psi}^2 + 2Z_b Z_h \cos(\psi) \dot{\theta}^2 + Z_h^2 \dot{\theta}^2)}{2} + \\
& \frac{m_n (X_n^2 \dot{\theta}^2 + Z_n^2 \dot{\theta}^2)}{2} + \frac{(J_{yyb} \cos^2(\psi) \dot{\theta} + J_{zzb} \sin^2(\psi) \dot{\theta}) \dot{\theta}}{2}
\end{aligned}$$

Equations of Motion:

$$\left[-J_{yyb} \sin(\psi) \cos(\psi) \dot{\psi} \dot{\theta} + \frac{J_{yyb} \cos^2(\psi) \ddot{\theta}}{2} + J_{yyb} \ddot{\theta} + \frac{J_{zzb} \sin^2(\psi) \ddot{\theta}}{2} + J_{zzb} \sin(\psi) \cos(\psi) \dot{\psi} \dot{\theta} + M_\phi Z_h \ddot{q} - M_{rphi} \sin(\psi) \dot{\psi} \dot{q} + \right.$$

```
[7]: # Output
M = sp.simplify(sp.trigsimp(LM.mass_matrix))      # symbolic mass matrix M(q)
F = sp.simplify(sp.trigsimp(LM.forcing))          # symbolic forcing vector
↳ F(q, qdot)
E = sp.simplify(sp.trigsimp(LM.eom))              # symbolic equations of
↳ motion E(q, qdot, qddot) = 0
print("Mass Matrix M:")
display(M)
print("Forcing Vector F:")
display(F)
print("Equations of Motion:")
display(E)
```

Mass Matrix M:

$$\begin{bmatrix} J_{yyb} \cos^2(\psi) + J_{yy n} + J_{zzb} \sin^2(\psi) + m_b (X_h^2 + Z_b^2 \cos^2(\psi) + 2Z_b Z_h \cos(\psi) + Z_h^2) + m_n (X_n^2 + Z_n^2) & X_h Z_b m_b \sin(\psi) & -X_h Z_b m_b \sin(\psi) \\ X_h Z_b m_b \sin(\psi) & J_{xxb} + Z_b^2 m_b & 0 \\ M_\phi Z_h + M_{rphi} \cos(\psi) & 0 & 0 \end{bmatrix}$$

Forcing Vector F:

$$\begin{bmatrix} J_{yyb} \sin(2\psi) \dot{\psi} \dot{\theta} - J_{zzb} \sin(2\psi) \dot{\psi} \dot{\theta} + M_{rphi} \sin(\psi) \dot{\psi} \dot{q} - X_h Z_b m_b \cos(\psi) \dot{\psi}^2 + X_h g m_b \cos(\theta) + X_n g m_n \cos(\theta) + Z_b^2 m_b \dot{\theta}^2 \\ (-J_{yyb} \cos(\psi) \dot{\theta}^2 + J_{zzb} \cos(\psi) \dot{\theta}^2 - M_{rphi} \dot{q} \dot{\theta} - Z_b^2 m_b \dot{\theta}^2) \\ -K_b q + M_{rphi} \dot{q} \end{bmatrix}$$

Equations of Motion:

$$\begin{bmatrix} -J_{yyb} \sin(2\psi) \dot{\psi} \dot{\theta} + \frac{J_{yyb} \cos(2\psi) \ddot{\theta}}{2} + \frac{J_{yyb} \ddot{\theta}}{2} + J_{yy n} \ddot{\theta} + J_{zzb} \sin(2\psi) \dot{\psi} \dot{\theta} - \frac{J_{zzb} \cos(2\psi) \ddot{\theta}}{2} + \frac{J_{zzb} \ddot{\theta}}{2} + M_\phi Z_h \ddot{q} - M_{rphi} \sin(\psi) \dot{\psi} \dot{q} \\ 0 \\ 0 \end{bmatrix}$$

```
[8]: params = {
    g: 9.81,
    m_n: 50000,
    m_b: 10000,

    J_xx_n: 1e6,
    J_yy_n: 5e6,
    J_zz_n: 1e6,

    J_xx_b: 1e5,
    J_yy_b: 1e6,
    J_zz_b: 1e6,

    X_n: -1.5,
    Z_n: 2.0,
    X_h: -2.0,
    Z_h: 3.0,
```

```

Z_b: 1.5,

k_t: 1e7,
c_t: 5e5,

GM_b: 8000,
K_b: 1e7,
M_phi: 5000,
M_rphi: 60000
}

```

```

[9]: # =====
# Numerical Mass and Forcing Matrices
# =====
M_sub = LM.mass_matrix.subs(params)
F_sub = LM.forcing.subs(params)

# State vector
states = [theta, psi, q, theta.diff(), psi.diff(), q.diff()]

# Lambdified functions
M_func = sp.lambdify((states,), M_sub, modules='numpy')
F_func = sp.lambdify((states,), F_sub, modules='numpy')

# =====
# 3-DOF Nonlinear ODE
# =====
def ode_3dof(t, y):

    # Evaluate matrices
    M_num = M_func(y)
    F_num = F_func(y)

    # Solve for accelerations
    accels = np.linalg.solve(M_num, F_num).flatten()

    return [
        y[3],          # theta_dot
        y[4],          # psi_dot
        y[5],          # q_dot
        accels[0],      # theta_ddot
        accels[1],      # psi_ddot
        accels[2]       # q_ddot
    ]

```

```

[10]: # =====
# Initial Conditions

```

```

# =====
# [theta, psi, q, theta_dot, psi_dot, q_dot]
init_3dof = [
    np.deg2rad(5),      # 5 deg tilt
    0.0,                # azimuth
    0.1,                # flap displacement
    0.0,                # tilt rate
    1.5,                # rotor speed
    0.0                 # flap velocity
]

# =====
# Run Simulation
# =====
t_span = (0, 300)
t_eval = np.linspace(*t_span, 1000)

sol = solve_ivp(ode_3dof, t_span, init_3dof, t_eval=t_eval, method='RK45')

# =====
# Print Simulation Information
# =====
print("Simulation Complete")
print(f"Number of Time Steps: {len(sol.t)}")
print(f"Final Tilt Angle: {np.rad2deg(sol.y[0,-1]):.3f} deg")
print(f"Final Rotor Speed: {sol.y[4,-1]:.3f} rad/s")
print(f"Final Blade Flap: {sol.y[2,-1]:.3f} m")

# =====
# Plot Results
# =====
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

# Tilt response
ax1.plot(sol.t, np.rad2deg(sol.y[0]), 'b')
ax1.set_ylabel(r'Tilt $\theta$ (deg)')
ax1.grid(True)

# Rotor speed
ax2.plot(sol.t, sol.y[4], 'r')
ax2.set_ylabel(r'Speed $\dot{\psi}$ (rad/s)')
ax2.grid(True)

# Blade flap response
ax3.plot(sol.t, sol.y[2], 'g')
ax3.set_ylabel(r'Blade Flap $q$ (m)')
ax3.set_xlabel('Time (s)')

```

```

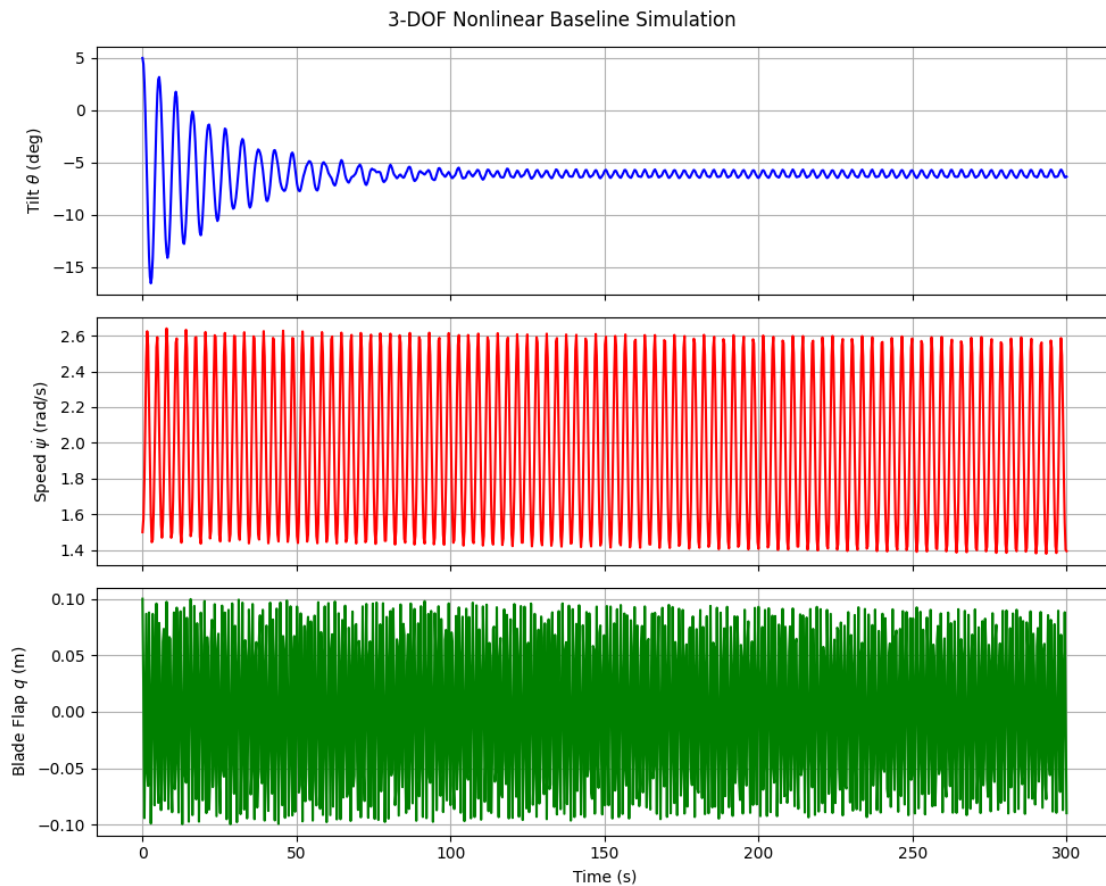
ax3.grid(True)

plt.suptitle('3-DOF Nonlinear Baseline Simulation')

plt.tight_layout()
plt.show()

```

Simulation Complete
 Number of Time Steps: 1000
 Final Tilt Angle: -6.343 deg
 Final Rotor Speed: 1.394 rad/s
 Final Blade Flap: -0.090 m



```

[11]: # =====
# Parameters for 3-DOF
# =====

# 2. Lambdify the 3rd DOF
M_sub = LM.mass_matrix.subs(params)

```

```

F_sub = LM.forcing.subs(params)

# Define the 6-element state vector: [pos1, pos2, pos3, vel1, vel2, vel3]
states = [theta, psi, q, theta.diff(), psi.diff(), q.diff()]

M_func = sp.lambdify((states,), M_sub, modules='numpy')
F_func = sp.lambdify((states,), F_sub, modules='numpy')

# =====
# 3. 3-DOF ODE Function
# =====
def ode_3dof(t, y):
    # y = [theta, psi, q, theta_dot, psi_dot, q_dot]

    # Solve  $M(q) * \ddot{q} = F(q, \dot{q})$ 
    M_num = M_func(y)
    F_num = F_func(y)

    # Extract accelerations [theta_ddot, psi_ddot, q_ddot]
    accels = np.linalg.solve(M_num, F_num).flatten()

    # Return [vel1, vel2, vel3, acc1, acc2, acc3]
    return [y[3], y[4], y[5], accels[0], accels[1], accels[2]]

# =====
# 4. Run Simulation
# =====
# Init: 5 deg tilt, 0 azimuth, 0.1m deflection, 0 tilt rate, 1.5 rad/s spin, 0
# ↪ flap rate
init_3dof = [np.deg2rad(5), 0.0, 0.1, 0.0, 1.5, 0.0]

t_span = (0, 300)
t_eval = np.linspace(t_span[0], t_span[1], 1000)

sol = solve_ivp(ode_3dof, t_span, init_3dof, t_eval=t_eval, method='RK45')

# =====
# 5. Visualizing the Baseline
# =====
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

ax1.plot(sol.t, np.rad2deg(sol.y[0]), 'b')
ax1.set_ylabel('Tilt  $\theta$  (deg)')

ax2.plot(sol.t, sol.y[4], 'r')
ax2.set_ylabel('Speed  $\dot{\psi}$  (rad/s)')

```

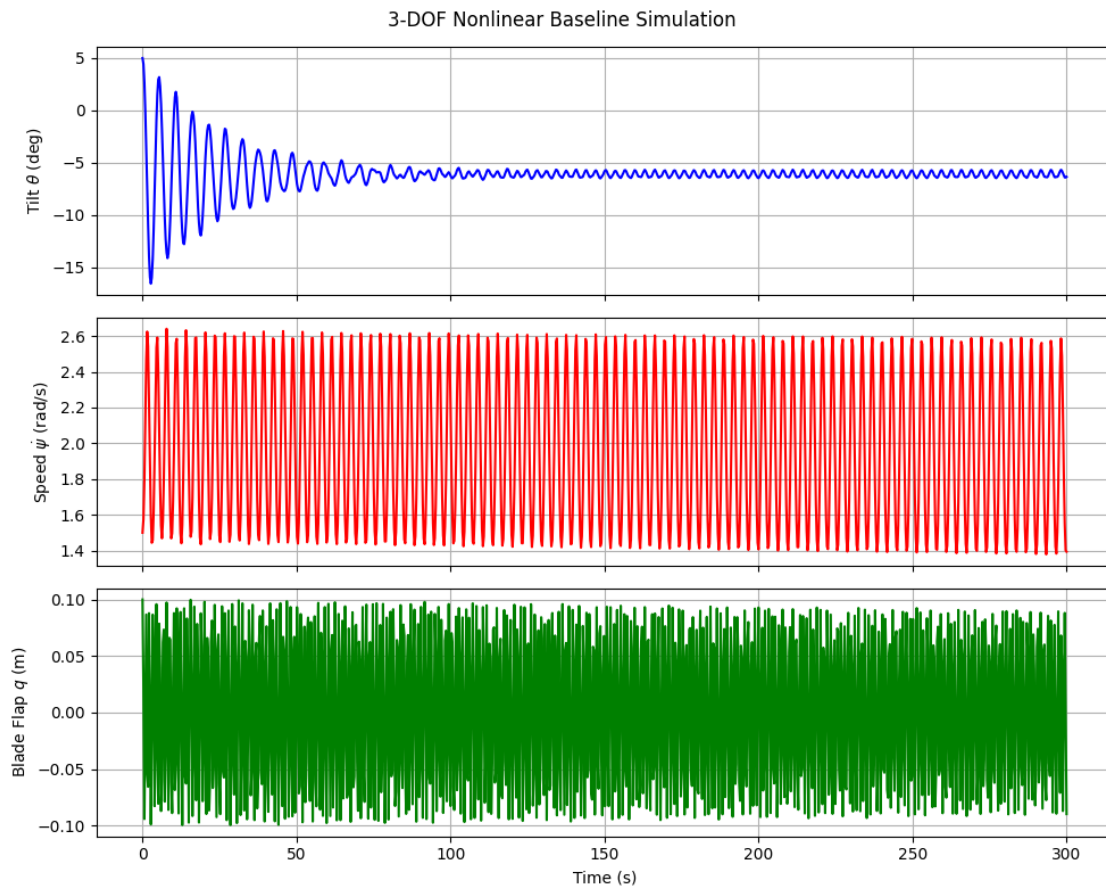


```

ax3.plot(sol.t, sol.y[2], 'g')
ax3.set_ylabel('Blade Flap  $q$  (m)')
ax3.set_xlabel('Time (s)')

for ax in [ax1, ax2, ax3]: ax.grid(True)
plt.suptitle('3-DOF Nonlinear Baseline Simulation')
plt.tight_layout()
plt.show()

```



1.1.1 Linearize

```

[12]: # Steady-State Operating Point

Omega = sp.Symbol('Omega', real=True, positive=True)
q0 = sp.Symbol('q0', real=True)

# Operating point
op_point = {
    theta: 0,

```

```

    theta.diff(t): 0,
    psi.diff(t): Omega,
    q: q0,
    q.diff(t): 0
}

# State Vectors
q_vec = sp.Matrix([theta, psi, q])
qd_vec = sp.Matrix([theta.diff(t), psi.diff(t), q.diff(t)])

# Linearized Matrices
forcing = LM.forcing

# Mass matrix
M_lin = sp.simplify(LM.mass_matrix.subs(op_point))
# Damping / gyroscopic matrix
C_lin = sp.simplify(-forcing.jacobian(qd_vec).subs(op_point))
# Stiffness / centrifugal matrix
K_lin = sp.simplify(-forcing.jacobian(q_vec).subs(op_point))

# Print Results
print("Linearized Matricies")
print("\nMass Matrix:")
display(M_lin)
print("\nDamping / Gyroscopic Matrix:")
display(C_lin)
print("\nStiffness / Centrifugal Matrix:")
display(K_lin)

# # Matrix Shape Check
# print("\nMatrix Dimensions:")
# print(f"M_lin: {M_lin.shape}")
# print(f"C_lin: {C_lin.shape}")
# print(f"K_lin: {K_lin.shape}")

```

Linearized Matricies

Mass Matrix:

$$\begin{bmatrix} J_{yyb} \cos^2(\psi) + J_{yy n} + J_{zzb} \sin^2(\psi) + m_b (X_h^2 + Z_b^2 \cos^2(\psi) + 2Z_b Z_h \cos(\psi) + Z_h^2) + m_n (X_n^2 + Z_n^2) & X_h Z_b m_b \sin(\psi) \\ X_h Z_b m_b \sin(\psi) & J_{xxb} + Z_b^2 m_b \\ M_\phi Z_h + M_{rphi} \cos(\psi) & 0 \end{bmatrix}$$

Damping / Gyroscopic Matrix:

$$\begin{bmatrix} -J_{yyb} \Omega \sin(2\psi) + J_{zzb} \Omega \sin(2\psi) - 2\Omega Z_b m_b (Z_b \cos(\psi) + Z_h \sin(\psi)) + c_t & 2\Omega X_h Z_b m_b \cos(\psi) & -M_{rphi} \Omega \sin(\psi) \\ 0 & 0 & 0 \\ -M_{rphi} \Omega \sin(\psi) & 0 & 0 \end{bmatrix}$$

Stiffness / Centrifugal Matrix:

$$\begin{bmatrix} -Z_n g m_n - g m_b (Z_b \cos(\psi) + Z_h) + k_t & -\Omega^2 X_h Z_b m_b \sin(\psi) & 0 \\ 0 & -Z_b g m_b \cos(\psi) & 0 \\ 0 & 0 & K_b \end{bmatrix}$$

1.1.2 Eigenvalue Analysis

```
[13]: # State-space A matrix construction

# Define the operating point dictionary
op_dict = {
    theta: 0,
    theta.diff(t): 0,
    theta.diff(t, 2): 0,
    psi.diff(t): Omega,
    psi.diff(t, 2): 0,
    q: q0,
    q.diff(t): 0,
    q.diff(t, 2): 0
}

# 2. Extract the un-inverted matrices (A_and_B=False is the default)
M_sym, A_sym, B_sym, _ = LM.linearize(
    q_ind=[theta, psi, q],
    qd_ind=[theta.diff(t), psi.diff(t), q.diff(t)],
    op_point=op_dict
)

subs_dict = {**params, psi: 0, Omega: Omega}

M_sub = M_sym.subs(subs_dict)
A_sub = A_sym.subs(subs_dict)

# 4. Create the final State-Space A matrix safely
q0_val = 0
A_state_space_3dof = sp.expand(M_sub.inv() * A_sub).evalf()

print("Flexible Blade, Tilting Nacelle State-Space A Matrix:")
display(A_state_space_3dof)
```

Flexible Blade, Tilting Nacelle State-Space A Matrix:

0	0	0	1.0	0
0	0	0	0	1.0
0	0	0	0	0
-1.46577806258678	0	16.0205062479974	-0.0854426999893197	0.0102531239987184Ω
0	1.20122448979592	0	0	0
13.741669336751	0	-1400.19224607498	0.801025312399872	-0.0961230374879846Ω

```
[14]: # -----
# Evaluate at a specific rotor speed
# -----
OMEGA_VAL = 1.5 # rad/s (~14 RPM)
A_evaluated = A_state_space_3dof.subs(Omega, OMEGA_VAL)

# Convert to NumPy
A_num = np.array(A_evaluated.tolist(), dtype=np.float64)

# Calculate eigenvalues and eigenvectors
eigenvalues, eigenvectors = la.eig(A_num)

# DOF index -> name mapping (positions 0-2 correspond to theta, psi, q)
dof_names = {0: "Nacelle Tilt", 1: "Rotor Azimuth", 2: "Blade Flapwise Bending"}

print("=== 3-DOF Eigenvalue Analysis Results ===")
print(f"Operating Speed: Omega = {OMEGA_VAL} rad/s\n")

for i, lam in enumerate(eigenvalues):
    # Filter numerical noise
    if np.abs(lam) < 1e-4:
        continue

    if np.abs(np.imag(lam)) > 1e-4:
        # Oscillatory mode - print positive-imaginary half only
        if np.imag(lam) > 0:
            wn_rad = np.abs(lam)
            wn_hz = wn_rad / (2 * np.pi)
            zeta = -np.real(lam) / wn_rad

            # Use eigenvector participation to name the mode
            dominant_dof = np.argmax(np.abs(eigenvectors[:, i])) % 3
            mode_name = dof_names.get(dominant_dof, "Unknown")

            print(f"Mode: {mode_name}")
            print(f" Eigenvalue (λ): {np.real(lam):.4f} ± {np.imag(lam):.4f}j")
            print(f" Natural Frequency (ω_n): {wn_hz:.4f} Hz ({wn_rad:.4f} rad/
→s)")

            print(f" Damping Ratio (ζ): {zeta:.4f}")
            print("-" * 40)
```

```

else:
    # Real root - only print once (positive root of the ± pair)
    if np.real(lam) > 0:
        dominant_dof = np.argmax(np.abs(eigenvectors[:, i])) % 3
        mode_name = dof_names.get(dominant_dof, "Unknown")

        print(f"Mode: {mode_name} (Non-Oscillatory)")
        print(f" Eigenvalue (λ): {lam:.6f}")
        print(f" Time Constant (τ): {1/np.abs(np.real(lam)):.4f} seconds")
        status = "UNSTABLE (Exponential Divergence)" if np.real(lam) > 0
    else "STABLE (Exponential Decay)"
        print(f" STATUS: {status}")
        print("-" * 40)

```

=== 3-DOF Eigenvalue Analysis Results ===

Operating Speed: $\Omega = 1.5$ rad/s

Mode: Blade Flapwise Bending

Eigenvalue (λ): $-0.0046 \pm 37.4212j$

Natural Frequency (ω_n): 5.9558 Hz (37.4212 rad/s)

Damping Ratio (ζ): 0.0001

Mode: Nacelle Tilt

Eigenvalue (λ): $-0.0381 \pm 1.1432j$

Natural Frequency (ω_n): 0.1821 Hz (1.1439 rad/s)

Damping Ratio (ζ): 0.0333

Mode: Rotor Azimuth (Non-Oscillatory)

Eigenvalue (λ): $1.096004 + 0.000000j$

Time Constant (τ): 0.9124 seconds

STATUS: UNSTABLE (Exponential Divergence)

1.1.3 Simulate Linearized Model

```

[15]: # -----
# Shared rotor speed and steady-state deflection
# -----
OMEGA_VAL = 1.5 # rad/s - must match eigenvalue section
QO_VAL    = 0.0 # steady-state blade deflection (m)

# Define the operating point - spinning rotor, consistent with eigenvalue section
op_dict = {
    theta: 0,
    theta.diff(t): 0,
    theta.diff(t, 2): 0,
    psi.diff(t): OMEGA, # spinning, not stopped
    psi.diff(t, 2): 0,

```

```

    q: q0,
    q.diff(t): 0,
    q.diff(t, 2): 0
}

# Single unified parameter dict (same as used throughout the notebook)

# Ensure EOM are fully formed before linearizing
LM.form_lagranges_equations()

# Extract un-inverted matrices (avoids SymPy hanging on symbolic inversion)
M_sym, A_sym_uninv, B_sym, _ = LM.linearize(
    q_ind=[theta, psi, q],
    qd_ind=[theta.diff(t), psi.diff(t), q.diff(t)],
    op_point=op_dict
)

# Substitute everything: parameters + psi=0 + Omega + q0
subs_dict = {**params, psi: 0, Omega: OMEGA_VAL, q0: Q0_VAL}
M_sub = M_sym.subs(subs_dict)
A_sub = A_sym_uninv.subs(subs_dict)

# Convert to NumPy float arrays
M_num = np.array(M_sub.evalf().tolist(), dtype=np.float64)
A_num_uninv = np.array(A_sub.evalf().tolist(), dtype=np.float64)

# Final state-space A matrix:  $A = M^{-1} * A_{uninv}$ 
A_num = la.inv(M_num) @ A_num_uninv

# -----
# Stability analysis
# -----
eigenvalues, eigenvectors = la.eig(A_num)

print("--- Linearized System Stability Analysis ---")
print(f"Operating Speed: Omega = {OMEGA_VAL} rad/s\n")

for i, lam in enumerate(eigenvalues):
    real_part = 0.0 if np.abs(np.real(lam)) < 1e-10 else np.real(lam)

    print(f"Mode {i+1}:")
    print(f"  Eigenvalue ( $\lambda$ ): {lam:.4f}")
    if real_part > 0:
        print(f"    Time Constant ( $\tau$ ): {1/np.abs(real_part):.4f} seconds")
        print("    STATUS: UNSTABLE (Exponential Divergence)")
    elif real_part < 0:
        print(f"    Time Constant ( $\tau$ ): {1/np.abs(real_part):.4f} seconds")

```

```

        print(" STATUS: STABLE (Exponential Decay)")
    else:
        print(" STATUS: marginally stable (Oscillatory)")

# -----
# Simulate the linearized model
# -----
# x represents PERTURBATIONS from the operating point:
# x = [ $\Delta\theta$ ,  $\Delta\psi$ ,  $\Delta q$ ,  $\Delta\theta$ ,  $\Delta\psi$ ,  $\Delta q$ ]
x0 = np.array([0.05, 0.0, 0.1, 0.0, 0.0, 0.0])
t_span = (0, 300)
t_eval = np.linspace(t_span[0], t_span[1], 500)

def lin_sys_dynamics(t, x):
    return A_num @ x

sol = solve_ivp(lin_sys_dynamics, t_span, x0, t_eval=t_eval, method='RK45')

# -----
# Reconstruct absolute states from perturbations + operating point
# x_absolute = x_operating_point +  $\Delta x$ 
# -----
theta_abs = sol.y[0, :] # op point = 0, so  $\Delta\theta = \theta_{abs}$ 
psi_dot_abs = OMEGA_VAL + sol.y[4, :] #  $\Omega + \Delta\psi$  (Speed perturbation is  $\psi$ 
    ↳ index 4)
q_abs = Q0_VAL + sol.y[2, :] #  $q0 + \Delta q$ 

# -----
# Plot
# -----
fig, axs = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

axs[0].plot(sol.t, np.rad2deg(theta_abs), color='b', label=' $\theta$  - Nacelle Tilt')
axs[0].set_ylabel('Angle [deg]')
axs[0].legend()
axs[0].grid(True)

# Updated Axis: Plotting absolute rotor speed instead of position
axs[1].plot(sol.t, psi_dot_abs, color='r', label=' $\omega$  - Rotor Speed (absolute)')
axs[1].set_ylabel('Speed [rad/s]')
axs[1].legend()
axs[1].grid(True)

axs[2].plot(sol.t, q_abs, color='g', label='q - Blade Deflection')
axs[2].set_ylabel('Deflection [m]')
axs[2].set_xlabel('Time [s]')
axs[2].legend()

```

```
axs[2].grid(True)

plt.suptitle(f'Linearized Model Response ( $\Omega$  = {OMEGA_VAL} rad/s)')
plt.tight_layout()
plt.show()
```

--- Linearized System Stability Analysis ---
Operating Speed: Ω = 1.5 rad/s

Mode 1:

Eigenvalue (λ): -0.0046+37.4212j
Time Constant (τ): 217.8136 seconds
STATUS: STABLE (Exponential Decay)

Mode 2:

Eigenvalue (λ): -0.0046-37.4212j
Time Constant (τ): 217.8136 seconds
STATUS: STABLE (Exponential Decay)

Mode 3:

Eigenvalue (λ): -0.0381+1.1432j
Time Constant (τ): 26.2259 seconds
STATUS: STABLE (Exponential Decay)

Mode 4:

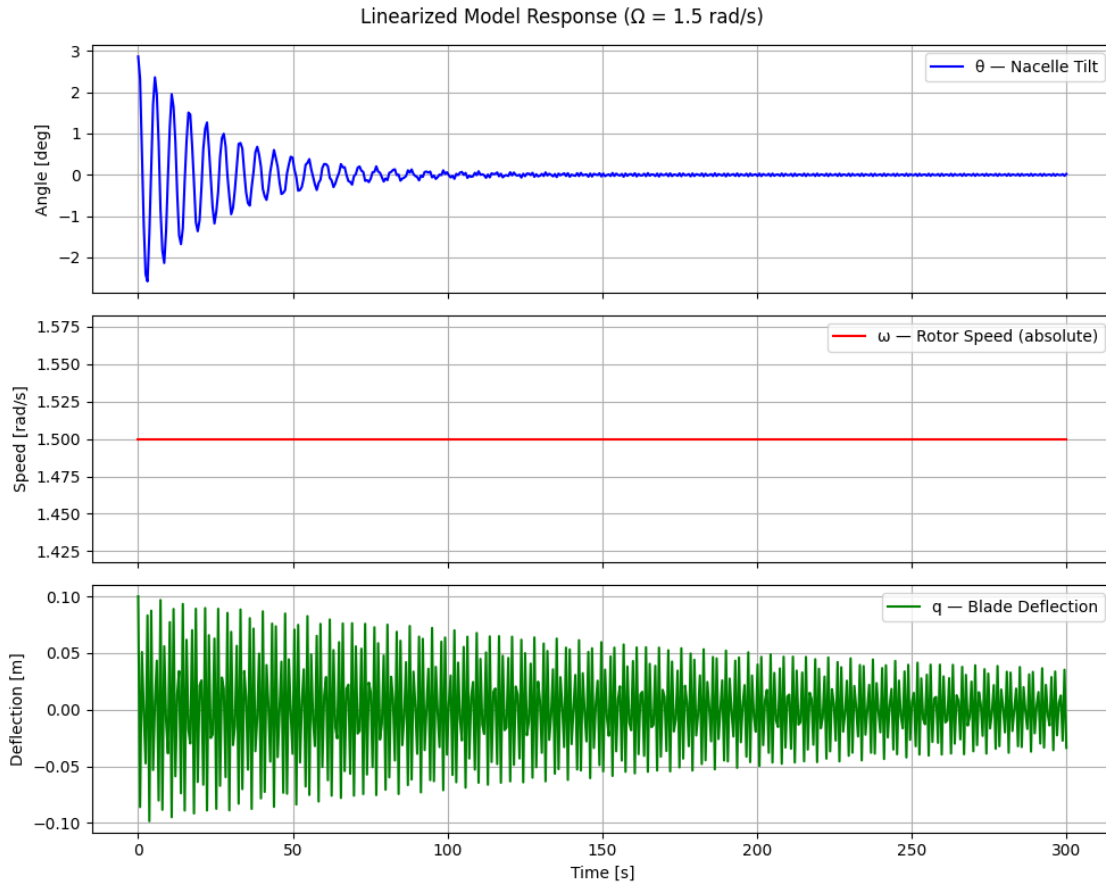
Eigenvalue (λ): -0.0381-1.1432j
Time Constant (τ): 26.2259 seconds
STATUS: STABLE (Exponential Decay)

Mode 5:

Eigenvalue (λ): 1.0960+0.0000j
Time Constant (τ): 0.9124 seconds
STATUS: UNSTABLE (Exponential Divergence)

Mode 6:

Eigenvalue (λ): -1.0960+0.0000j
Time Constant (τ): 0.9124 seconds
STATUS: STABLE (Exponential Decay)



1.1.4 Validate Linearization with Integration

```
[16]: # =====
# Phase 3: Validate Linearization via Integration (Transient Overlap)
# =====

# 1. Define the Linear ODE Function
def ode_linear_3dof(t, x_delta):
    # dx/dt = A * x
    # x_delta is the perturbation state vector:
    # [delta_theta, delta_psi, delta_q, delta_theta_dot, delta_psi_dot,
    # ↪ delta_q_dot]
    return A_num @ x_delta

# 2. Define Initial Conditions (1-degree tilt perturbation)
theta_pert = np.deg2rad(1.0)

# Linear Initial Conditions (Perturbations from equilibrium)
# x_delta_0 = [th, psi, q, th_d, psi_d, q_d]
```

```

init_linear = [theta_pert, 0.0, 0.0, 0.0, 0.0, 0.0]

# Nonlinear Initial Conditions (Absolute states)
# We add the perturbation to the steady-state operating point
# Assuming  $q_0 = 0$  for the steady-state deflection in this conservative model
init_nonlinear = [theta_pert, 0.0, 0.0, 0.0, OMEGA_VAL, 0.0]

# 3. Setup Simulation Time
t_span = (0, 1) # Simulate for 1.5 seconds to capture the transient overlap and
↳divergence
t_eval = np.linspace(t_span[0], t_span[1], 500)

# 4. Integrate Both Models
print("Simulating Nonlinear Baseline...")
sol_nonlin = solve_ivp(ode_3dof, t_span, init_nonlinear, t_eval=t_eval,
↳method='RK45')

print("Simulating Linearized State-Space...")
sol_lin = solve_ivp(ode_linear_3dof, t_span, init_linear, t_eval=t_eval,
↳method='RK45')

# 5. Convert Linear Perturbations back to Absolute States for Comparison
# Total State = Operating Point + Perturbation
theta_lin_abs = np.rad2deg(sol_lin.y[0] + 0.0)
psi_d_lin_abs = sol_lin.y[4] + OMEGA_VAL
q_lin_abs = sol_lin.y[2] + 0.0 # Add  $q_0$  here if you calculated a non-zero
↳steady-state deflection

# Convert Nonlinear States
theta_nonlin_abs = np.rad2deg(sol_nonlin.y[0])
psi_d_nonlin_abs = sol_nonlin.y[4]
q_nonlin_abs = sol_nonlin.y[2]

# 6. Plot the Validation
fig, axs = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

# Nacelle Tilt
axs[0].plot(sol_nonlin.t, theta_nonlin_abs, 'k-', lw=2, label='Nonlinear
↳Baseline')
axs[0].plot(sol_lin.t, theta_lin_abs, 'b--', lw=2, label='Linear LTI (Frozen
↳Azimuth)')
axs[0].set_ylabel('Nacelle Tilt [deg]')
axs[0].legend()

# Rotor Speed
axs[1].plot(sol_nonlin.t, psi_d_nonlin_abs, 'k-', lw=2)

```

```

axs[1].plot(sol_lin.t, psi_d_lin_abs, 'r--', lw=2)
axs[1].set_ylabel('Rotor Speed [rad/s]')

# Blade Flapwise Deflection
axs[2].plot(sol_nonlin.t, q_nonlin_abs, 'k-', lw=2)
axs[2].plot(sol_lin.t, q_lin_abs, 'g--', lw=2)
axs[2].set_ylabel('Blade Flap Deflection [m]')
axs[2].set_xlabel('Time [s]')

for ax in axs:
    ax.grid(True)

plt.suptitle('Phase 3 Validation: Transient Overlap (1° Tilt Perturbation)')
plt.tight_layout()
plt.show()

```

Simulating Nonlinear Baseline...

Simulating Linearized State-Space...

